

第 0 章 跑通第一个程序

这一章从零开始。目标很简单，把 Python 装到你的电脑上，然后写出并运行你的第一个程序。装环境是最容易劝退新手的一步，所以这一章讲得细一些，包括装完怎么验证、报错了怎么办。这些都弄好之后，后面几章就可以专心写代码了。

0.1 先让电脑说句话

本节任务：装好 Python，让它输出一句话，再用它当计算器算点东西。

0.1.1 第一步，把 Python 装上

Python 不是电脑自带的，得先装。去官网 python.org 的下载页，它会自动认出你的系统，点那个大按钮下载最新版就行。

Windows 用户装的时候有一处要特别注意。安装界面最下面有一个写着 Add python.exe to PATH 的勾选框，**一定要勾上**再点安装。勾了它，你才能在任意位置直接敲 python 这个命令。忘了勾也不要紧，重新运行一遍安装程序，选修改，把它补上即可。

macOS 用户直接下载安装包一路点下去就行。Linux 多数发行版自带 Python，可以先跳到下一步验证一下。

0.1.2 第二步，验证装好了没有

打开命令行。Windows 上按下 Win 加 R，输入 cmd 回车；macOS 上打开终端这个程序。然后输入下面这行并回车：

```
python --version
```

如果看到类似下面的输出，说明装好了，数字是几都没关系，只要是 3 开头：

```
Python 3.13.5
```

0.1.3 第三步，说第一句话

在命令行里直接输入 python 并回车，你会看到几行版本信息，然后出现三个大于号：

```
>>>
```

这就是 Python 在等你输入。试着敲下面这行再回车：

```
>>> print("你好")
你好
```

电脑把这句话打了出来。再试试让它算点东西，输入算式回车就有结果：

```
>>> 2 + 3
5
>>> 65 * 12 + 8
788
```

到这里，Python 已经能用了。想退出这个状态，输入 `exit()` 回车即可。

下面把这几步背后的事情讲清楚。

0.2 Python 到底是什么

本节任务：搞清你刚装的东西是什么，它是怎么把你写的字变成电脑动作的。

你写的 `print("你好")` 是给人看的文字，电脑并不直接懂。中间需要一个翻译，一句一句地读你的代码、照着做，这个翻译就是 **解释器**。你刚才装的 Python，主要就是这个解释器程序。

所以写 Python 程序的过程是这样的，你把想让电脑做的事按 Python 的语法写下 **★核心** 来，解释器读一句执行一句。这也是它叫 **解释型语言** 的原因，不需要先整体翻译成另一种文件再运行，写完直接就能跑，改一行也能立刻看到效果。这个特点让它特别适合边学边试。

0.3 两种用法，交互和脚本

本节任务：知道什么时候在命令行里直接敲，什么时候该写成文件。

上一节那种敲一行、立刻出结果的方式，叫 **交互模式**。它的提示符是三个大于号，敲什么立刻算什么，特别适合试探一个想法、验证一句话对不对。缺点是关掉窗口，写的东西就都没了。

真正要写一个完整程序，就得把代码存进文件，这叫 **脚本**。新建一个文本文件，名 **★核心** 字随便取，但扩展名要是 `py`，比如 `hello.py`，把代码写进去：

```
1 print("你好，Python")
2 print(2 + 3)
```

存好后，在命令行里先切换到这个文件所在的目录，再运行它：

```
python hello.py
```

```
你好, Python
```

```
5
```

注意脚本和交互模式有个区别。交互模式里你敲 $2 + 3$ 会自动显示 5，但在脚本里，只写一行 $2 + 3$ 屏幕上什么也不会出现，必须用 `print` 明确地要求它输出。这是新手最常见的困惑之一。

两种用法的对照如表 1。往后的练习都是脚本形式，因为要写的代码不止一行。

表 1: 交互模式和脚本的分工。

	交互模式	脚本
怎么进	命令行敲 <code>python</code>	写进 <code>py</code> 文件再运行
适合做什么	试一句、验证想法、当计算器	完整程序、要反复运行的东西
结果怎么出来	敲完自动显示	必须用 <code>print</code>
关掉之后	内容不保留	文件一直在

0.4 代码怎么写才不出错

本节任务：认识 Python 写代码的几条基本规矩，尤其是它很特别的缩进。

0.4.1 一行写一句

Python 一般一行写一条语句，写完直接换行，不需要在末尾加分号。

0.4.2 注释

在代码里加说明文字，用井号开头，井号后面那一行剩下的内容 Python 都不看，只给人读：

```
1 # 这一行是注释, Python 会跳过
2 print("你好") # 也可以写在语句后面
```

养成写注释的习惯，过几天回头看自己的代码时会感谢自己。

0.4.3 缩进决定层次

这是 Python 最有个性的地方。很多语言用大括号表示哪些代码属于一块，Python ★核心用**缩进**，也就是行首的空格。同一块里的代码要缩进相同的距离，标准是四个空格。

```
1 if 3 > 2:
```

```
2     print("这一行属于 if") # 缩进四格，是 if 的一部分
3     print("这一行也属于 if")
4 print("这一行不缩进，不管条件成立与否都会执行")
```

缩进不是排版好看，而是有实际含义。同一块里缩进不一致，程序会直接报错。这一点后面讲分支和循环时会反复用到，现在只要记住行首的空格有意义就够了。

0.4.4 大小写要分清

Python 区分大小写，`print` 和 `Print` 是两个不同的东西，后者会报错。

0.5 用什么写代码

本节任务：挑一个顺手的编辑器，别用记事本。

写一两行可以用交互模式，写完整程序需要一个编辑器。推荐 **VS Code**，免费、通用、装个 Python 扩展就能高亮代码、提示错误、一键运行。装好 VS Code 后，在扩展面板里搜 Python，装微软官方那个即可。

如果暂时不想装，Python 自带一个叫 IDLE 的简易编辑器，在开始菜单里能找到，写小程序够用。

不建议用记事本，它不会高亮代码，也容易在保存时把格式弄乱。

0.6 报错了怎么办

本节任务：新手最常撞见的几种报错，学会看懂它们。

写代码出错是常态，会看报错就成功了一半。报错信息可能有好几行，看不懂上面 ★核心的没关系，**最后一行才是关键**，它告诉你错在哪一类。

下面是三种最常撞见的。

第一种，语法写错了：

```
>>> print("你好"
SyntaxError: '(' was never closed
```

括号只开没关。`SyntaxError` 就是句子没写对，多半是括号引号不成对，或者少了冒号。

第二种，用了不存在的名字：

```
>>> print(nihao)
NameError: name 'nihao' is not defined
```

`NameError` 意思是这个名字它不认识。常见原因是拼错了，或者想写一段文字却忘了加引号。

第三种，缩进不对：

```
>>> if 3 > 2:
...     print("对")
...     print("错")
IndentationError: unindent does not match any outer indentation level
```

`IndentationError` 就是行首空格数量不一致。统一用四个空格就能避开大部分这类问题。

报错不可怕，它其实在帮你定位问题。遇到看不懂的报错，把最后一行原样搜索一下，基本都能找到答案。

0.7 小结

这一章把环境准备好了。Python 是一个解释器程序，读一句执行一句。它有两种用法，交互模式适合试，脚本适合写完整程序，脚本里必须用 `print` 才有输出。写代码要注意缩进有实际含义、大小写要分清。报错时先看最后一行。下一章开始正式写代码，从变量和基本类型讲起。

本章知识点

- **解释器** —— 读一句执行一句的翻译程序，你装的 Python 主要就是它
- **PATH** —— 系统找命令的路径清单，Windows 装 Python 时要勾上加入 PATH
- **交互模式** —— 命令行里敲 `python` 进入，提示符是三个大于号，敲一行出一行结果
- **脚本** —— 把代码存进扩展名为 `py` 的文件，用 `python` 文件名运行
- **print** —— 把结果输出到屏幕，脚本里不写它就看不到东西
- **注释** —— 井号开头的说明文字，只给人看，Python 会跳过
- **缩进** —— 行首的空格，决定哪些代码属于同一块，标准是四个空格
- **SyntaxError** —— 句子写错了，多半是括号引号不成对或少了冒号
- **NameError** —— 用了不认识的名字，多半是拼错或忘了加引号
- **IndentationError** —— 缩进不一致，统一四个空格可避开